

MFCENTRAL API SPECIFICATIONS & CAPITAL GAIN STATEMENT DOCUMENT

Contents

1. What is MFcentral? 1

2. Getting Started 1

3. Onboarding & API Invoking Process 1

3.1 Get Client ID and Client Secret 2

3.2 Generate Token 2

3.3 Submitting Request to MFC 3

3.4 Submitting Investor consent to MFCentral (Validating OTP) 7

3.5 Submit request to get Response <CGS document>: 8

Exceptions and errors 8

4. Capital Gain Statement API 9

5. Common HTTP error codes 12

6. Appendix 12

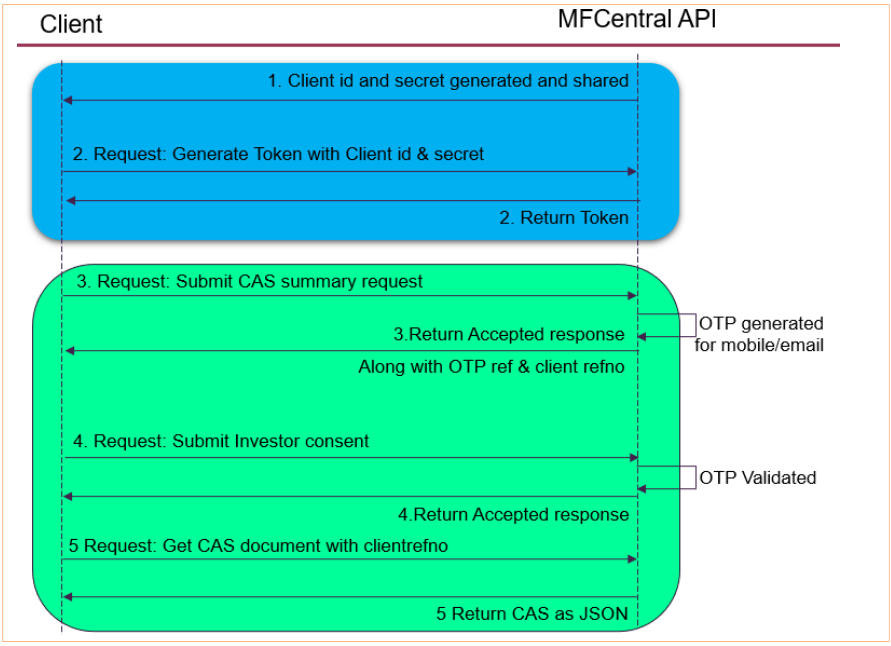
API Specification and Integration Document

1. What is MFcentral?

MFcentral is an application that allows users to seamlessly navigate, search, analyze, invest in MF trading platform for all funds serviced by RTA's. It lets you execute orders in real time, manage user portfolios and place any Non-Financial and Financial Transactions request across all Mutual Funds.

2. Getting Started

This application provides a single unified portal for the user to access information regarding investments made across mutual funds. This Walkthrough goes over the basics of the authentication process as it applies to the MFCentral API using Json Web Tokens (JWT).



Above illustration explained w.r.t GetStatementDocument API, the invoking flow remains the same for other APIs too.

3. Onboarding & API Invoking Process

To access MFcentral APIs, you will need the client Id and Client secret to generate the token.

You may contact MFcentral representative for generating Client Id and secret.

Once you have the credentials, connecting to our API is easy. You can proceed with generating the token, which should be passed in the header. Please refer to section 3.1

The following steps walk you through the basics to get started using the API.

3.1 Get Client ID and Client Secret.

You can get the client ID and secret, through an Offline process.

If you think the key has been compromised, contact immediately to generate a new Client Secret

```
Below details will be shared by MFCentral during on-boarding

        String clientId = "";
        String clientSecret = "";
        String userName = "";
        String password = "";
        String encryptionDecryptionKey = "";
        String privateKey = "";
        String publicKey = "";
```

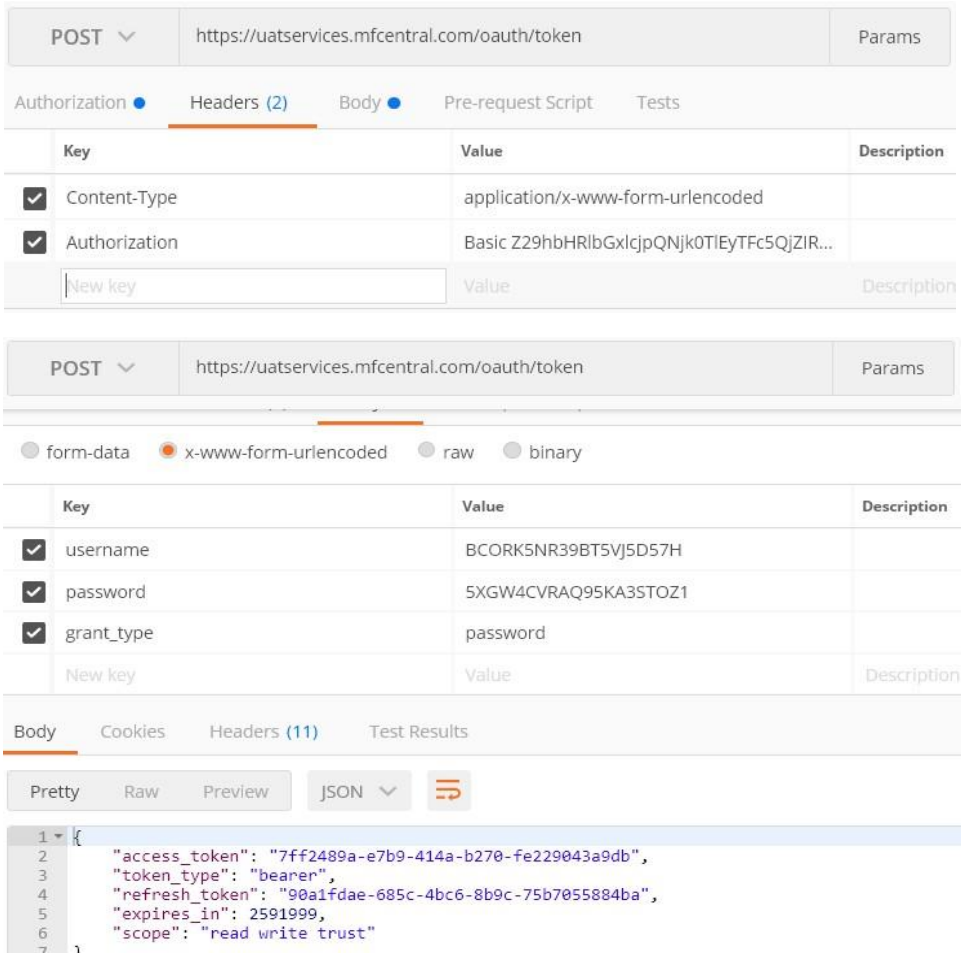
3.2 Generate Token

POST <baseURL>/oauth/token. This endpoint is accessible only over HTTPS. Plain HTTP connections are refused.

To generate a token that will be used for authorizing all subsequent APIs, pass the Client ID, Client Secret and in the Authorization header.

Token generation involves passing Headers and Body together to the URL. Refer to the below Postman screenshot on how to generate token.

Every token object will have an expiry time mentioned in seconds. The limit is set for 30 days. Ensure that you are creating a new token object if the existing token object is expired.



Sample:

Endpoint: https://uatservices.mfcentral.com/oauth/token

Headers:
Content-Type: application/x-www-form-urlencoded
Authorization: Basic dGVzdC1jbGllbnQxOnNzcGwkMTIzNA==
Body:
username: mfc-client1
password: Cams$1234
grant_type: password

Response:

```
{
   "access_token": "dd33510a-f161-4818-8d35-65a64f617516",
   "token_type": "bearer",
   "refresh_token": "a0f749fa-2f97-44fd-be93-a5da24a22ca0",
   "expires_in": 2591999,
   "scope": "read write trust"
}
```

```
For JAVA users - // Request 1 - Token Generation
    String tokenGenerationUrl = "https://uatservices.mfcentral.com/oauth/token";
String token = getToken(tokenGenerationUrl, clientId, clientSecret, userName,
password);
```

3.3 Submitting Request to MFC

The token is sent in the header (Authorization Tag) along with the keyword “Bearer”. This token is authenticated and on success the validation of the signature process begins. If authentication fails, the response is returned, and the process is terminated.

Sample: URL: https://uatservices.mfcentral.com/api/client/V1/<endpoint>

3.3.1) Request Before encryption:

```
{
  "clientRefNo": "testing12345",
  "pan": "AANPE5444K",
  "pekrn": "",
  "mobile": "+919550755111",
  "email": ""
}
```

Note: *clientRefNo should be unique across different unique requests* from client for tracking purpose

3.3.2) Encrypt the above request:

The above request should be encrypted using SHA 256 hash logic.
To encrypt/decrypt data, use the below details.
During Encryption: Final value should be base 64 encoded value from the encrypted result.
Take SHA-256 hash of the shared key and use the first 32 byte of the hash as your secret key for both encryption and decryption.

Algorithm	:	AES Symmetric 256-bit
Cipher Mode	:	CBC
Padding	:	PKCS5Padding
IV text	:	<Constant> - will be shared via email separately

Reference

Use CryptUtils.encryptDecrypt() for decryption

Sample Usage for Encryption :`CryptUtils.encryptDecrypt("ENCRYPT", payload, encryptionKey);`

3.3.3) Generate Signature for the above encrypted request

Generate a signature which should passed along with the request. The signed value is a JWS (JSON WEB SIGNATURE) based value. Use the below detail to generate a JWS.

Algorithm: RS256 (RSA using SHA-256 hash algorithm)

Serialization: Compact Serialization, detached payload

Header (base64url-encode payload): false

Reference:

Use generateSignature function

Sample Usage: generateSignature(encryptedText, privateKey);

```
public static String generateSignature(String payload, String privateKey) {
    JSONObject output = new JSONObject();

    try {
        RsaJsonWebKey rsaJsonWebKey = getPrivateKey(privateKey);
        JsonWebSignature jws = new JsonWebSignature();
        jws.setAlgorithmHeaderValue(AlgorithmIdentifiers.RSA_USING_SHA256);
        jws.setKeyIdHeaderValue(rsaJsonWebKey.getKeyId());
        jws.getHeaders().setObjectHeaderValue(HeaderParameterNames.BASE64URL_ENCODE_PAYLOAD, false);
        jws.setPayload(payload);
        jws.setKey(rsaJsonWebKey.getPrivateKey());

        String digitalSignature = jws.getDetachedContentCompactSerialization();

        output = new JSONObject();
        output.put("request", payload);
        output.put("signature", digitalSignature);

        return output.toString();
    } catch (Exception ex) {
        ex.printStackTrace();
    }

    return null;
}
```

```
public static RsaJsonWebKey getPrivateKey(String privateKey) throws Exception {
    JsonWebKeySet jsonWebKeySet = new JsonWebKeySet(privateKey);
    RsaJsonWebKey rsaJsonWebKey = (RsaJsonWebKey) jsonWebKeySet.getJsonWebKeys().get(0);
    return rsaJsonWebKey;
}
```

sample process – for JAVA based Users:

```
      // Inputs to the request

String pan = "ACXXXX91XX";
String pekrn = "";
String mobile = "+91XXXXX70XX9";
String email = "";
String clientReferenceNumber = "testing12345"; //dynamic value generated by
client

// Request 2 - submit pan/pekrn & mobile/email to MFCentral for CGS
generation.

UAT URL : https://uatservices.mfcentral.com/api/client/V1/submitstatementrequest

String request = "{\"clientRefNo\": " + "\"" + clientReferenceNumber + "\""
                                + ", \"pan\": " + "\"" + pan + "\""
                                + ", \"pekrn\": " + "\""  + pekrn + "\""
                                + ", \"mobile\": " + "\"" + mobile + "\""
                                + ", \"email\" : " + "\"" + email + "\""
                                + " }";

String cgsRequestUrl =
"https://uatservices.mfcentral.com/api/client/V1/submitstatementrequest "
JSONObject response = submitRequestToMFCentral(
cgsRequestUrl,
token,
clientId,
encryptionDecryptionKey,
privateKey, publicKey, request);
```

Response From MFC

```
{
      "reqId": 56475,
      "otpRef": "9b69c14e-76f5-4ccb-9cd1-22e20793c4e9",
      "userSubjectReference": "",
      "clientRefNo": "testing12345"
}
```

3.3.4) Submitting the request to MFCentral

Once you have generated the signature, submit your request in the below request format.

{"signature": “<signature generated in step 3.3.3>”,
“request”: “<encrypted request generated in step 3.3.2>”}

Headers:

Content-Type: application/json
ClientId: test-client1
Authorization: Bearer dd33510a-f161-4818-8d35-65a64f617516

Method: POST

Reference

```
HttpHeaders headers = new HttpHeaders();
headers.add("Content-Type", "application/json");
headers.add("ClientId", clientId);
headers.add("Authorization", "Bearer " + token);

RestTemplate restTemplate = new RestTemplate();
HttpEntity<String> request = new HttpEntity<String>(mfcRequest, headers);

JSONObject responseJson = null;
try {
    ResponseEntity<String> response = restTemplate.postForEntity( url, request, String.class );

    String responseBody = response.getBody();
    responseJson = new JSONObject(responseBody);
} catch (HttpStatusCodeException e)  {

    String responseBody = e.getResponseBodyAsString();
    responseJson = new JSONObject(responseBody);
}
```

Sample Response: {"signature":"","response":""}

3.3.5) Verifying the signature

MFCentral returns response payload along with the digital signature. Verify the signature before processing the response.

Reference:

Use verifySignature function
Sample Usage:

```
verifyDigitalSignature(response, signature, publicKey);
```

```
public static boolean verifyDigitalSignature(String payload, String digitalSignature, String publicKey) {

    try {
        RsaJsonWebKey rsaJsonWebKey = getPublicKey(publicKey);

        JsonWebSignature verifierJws = new JsonWebSignature();
        verifierJws.setAlgorithmConstraints(new AlgorithmConstraints(
                                    AlgorithmConstraints.ConstraintType.WHITELIST,
                                    AlgorithmIdentifiers.RSA_USING_SHA256));
        verifierJws.setKey(rsaJsonWebKey.getPublicKey());
        verifierJws.setCompactSerialization(digitalSignature);
        verifierJws.setPayload(payload);
        return verifierJws.verifySignature();

    } catch (Exception ex) {
        ex.printStackTrace();
    }

    return false;
}
```

3.3.6) Decrypt the response:

Before decryption, decode the base 64 value first.

Algorithm	:	AES Symmetric 256-bit
Cipher Mode	:	CBC
Padding	:	PKCS5Padding
IV text	:	<constant>

Reference

Use CryptUtils.encryptDecrypt() for decryption
Sample Usage:

```
CryptUtils.encryptDecrypt("DECRYPT", response, encryptionKey);
```

3.4 Submitting Investor consent to MFCentral (Validating OTP):

Upon successful authentication, you shall request for user OTP validation.

Once the User enters the OTP, MFcentral will validate and if validation is successful, shall allow to proceed with the next step, else return error as OTP validation failed.

For every request OTP validation shall be a mandatory process.

Below is the process for Investor consent

URL: https://uatservices.mfcentral.com/api/client/V1/investorconsent

Headers:

Content-Type: application/json
ClientId: test-client1
Authorization: Bearer dd33510a-f161-4818-8d35-65a64f617516

3.4.1) Request Before encryption:

```
{
    "reqId": 56475,
    "otpRef": "9b69c14e-76f5-4ccb-9cd1-22e20793c4e9",
    "userSubjectReference": "",
    "clientRefNo": " testing12345",
    "enteredOtp": "973947"
}
```

reqId – generated from response – step 3.3.6
otpRef – generated from response – step 3.3.6

perform steps 3.3.2 to 3.3.6 for the payload to be sent and to get the response

```
// Request 3 - Submit investor Consent to MFCentral

String otpValidationUrl =
"https://uatservices.mfcentral.com/api/client/V1/investorconsent";
JSONObject otpRequest = new
JSONObject("{\"reqId\":55718,\"otpRef\":\"35979fc2-e2c8-4faa-b6dc-
ab6505a4612e\",\"userSubjectReference\":\"\",\"clientRefNo\":\"testing12345
\"}");
otpRequest.put("enteredOtp", "574224"); // otp entered by user

submitRequestToMFCentral(
      otpValidationUrl, token, clientId, encryptionDecryptionKey,
privateKey,publicKey, otpRequest.toString());
```

Response:

Response from MFCentral: Accepted status 202 <<empty>>

3.5 Submit request to get Response <CGS document>:

Upon successful OTP validation, you can invoke the MFcentral APIs. The API response structure are provided in section 4

Below sample is provided for get CGS document, likewise you may use the respective service endpoints.

Endpoint: https://uatservices.mfcentral.com/api/client/V1/getStatementDocument

Headers:

Content-Type: application/json

ClientId: test-client1

Authorization: Bearer dd33510a-f161-4818-8d35-65a64f617516

3.5.1) Request Before encryption:

Use the reqId and clientRefNo sent in the previous request 3.4.1

{"reqId":56475,"clientRefNo":"testing12345"}

```
      // Request 4 - Get CGS Document

String cgsDocumentUrl =
"https://uatservices.mfcentral.com/api/client/V1/getStatementDocument"
 JSONObject cgsDocument = submitRequestToMFCentral(
                              cgsDocumentUrl, token,clientId,
                        encryptionDecryptionKey, privateKey, publicKey,
                              otpRequest.toString());

System.out.println("CGS: " + cgsDocument);
```

perform steps 3.3.2 to 3.3.6 for the payload to be sent and get the response

Note: As this is asynchronous call, There might be a slight delay in fetching the response and you might get the interim response as below:

```
{"reqId":"2437962","errors":[{"code":"422","message":"We are in process of
generating the Statement. Please visit after sometime."}]}
```

Code: 422 – represents that the JSON is not yet ready as it might take some time to get response from all AMC. Therefore, a slight delay is expected.

However, you can retry with the same request to get the successful response.

Exceptions and errors

A failure response is preceded by the corresponding 40x or 50x HTTP header. The code key in the response contains the value error. The message key contains a textual description of the error.

Below are the Possible error codes:

Status Code	Actual Status	Reason/Description
400	Valid Business error	If there are multiple request objects within the same request, you can know the actual status by invoking the Status API which has have the unsuccessful folio details
500	Internal server error or Connectivity error	In this case, MFcentral will retry again until a proper response is returned from the RTA. You can invoke the Status API to get the actual status.
401	Unauthorized	Invalid token or client Id

422	Unprocessable Entity (WebDAV)	Invalid Input or Request Format

Sample error Response:

```
{
  "code": "400",
  "message": "Error message",
}
```

4. CGS Statement API

Description: The MFCentral CGS API retrieves PAN level data of the Investor. Upon accepting the request with PAN and post validating the Mobile/email via OTP authentication, the service returns the data segmented into Summary level, Scheme level and Transaction level information. Retrieves data based on the transactions executed within given financial year.

URL: https://uatservices.mfcentral.com/api/client/V1/submitstatementrequest

Request:

Parameters	Required	Description
reqId	Yes	Unique request Id generated by Client
Pan	Yes	Pan of the Investor
Pekrn		
Email	Yes*	Valid email address
Mobile	Yes*	Valid mobile number
fromDate	Yes	Financial year beginning
toDate	Yes	Financial year end.

*either email or mobile is mandatory to validate the request

* API will not accept request if both email and Mobile are given

*The fromDate and toDate to be specified based on the current FY, Previous FY, 2nd Previous FY or 3rd Previous FY. At any point, the API will generate the statement for one FY only.

Note: Response at PAN level

REQUEST:

```
{
    "clientRefNo": "",
    "reqId": "0111-0001",
    "pan": "AXXPXXXXXN",
    "pekrn": "",
    "email": "",
    "mobile": "",
    "fromDate": "01-Apr-2022”,
    "toDate": "31-Mar-2023”
}
```

RESPONSE:

```
{
    "reqId": "19686169",
    "pan": "AXXPXXXXXN",
    "pekrn": "",
    "email": "",
    "mobile": "1234567890",
    "fromDate": "01-Apr-2022",
```

```
    "toDate": "01-Mar-2023",
    "data": [
        {
            "transactionDetails": [
                {
                    "amc": "D",
                    "folio": "2915704 / 57",
                    "schemeName": "DSP Tax Saver Fund - Direct Plan - Growth, ISIN
: INF740K01OK1",
                    "desc": "Purchase",
                    "date": "2/23/2015 12:00:00 AM",
                    "units": 154.097,
                    "amount": 2500.0,
                    "price": 80.757,
                    "stt": 0.03,
                    "desc_1": "Redemption",
                    "date_1": "7/13/2021 12:00:00 AM",
                    "purhUnit": 30.957,
                    "redUnits": 30.957,
                    "unitCost": 32.447,
                    "indexedCost": 0.0,
                    "grandfatheredUnits(As On 31/01/2018)": 30.957,
                    "grandfatheredNAV(As On 31/01/2018)": 50.116,
                    "grandfatheredMarketValue(As On 31/01/2018 )": 1551.44,
                    "shortTerm": 0.0,
                    "longTermWithIndex": 0.0,
                    "longTermWithoutIndex": 948.55,
                    "taxPerc": 0.0,
                    "taxDeduct": 0.0,
                    "taxSurcharge": 0.0
                }
            ],
            "schemeDetails": [
                {
                    "schemeName(Redeem/Switchout)": "DSP Tax Saver Fund -
Direct Plan - Growth, ISIN : INF740K01OK1 (ELSS)",
                    "totalCount": 1.0,
                    "totalAmount": 2500.0,
                    "totalCost": 1004.46,
                    "indexedCost": 0.0,
                    "marketValue( As On 31/01/2018)": 1551.44,
                    "shortTerm": 0.0,
                    "longTermWithIndex": 0.0,
                    "longTermWithoutIndexation": 948.55,
                    "tds": 0.0
                }
            ],
            "summaryDetails": [
                {
                    "summaryOfCapitalGains": "Full Value Consideration",
                    "01/04 to 15/06": 0.0,
                    "16/06 to 15/09": 0.0,
                    "16/09 to 15/12": 3932.36,
```

```
                                        "16/12 to 16/03": 0.0,
                                        "16/03 to 31/03": 0.0,
                                        "total": 3932.36
                                }
                        ],
                        "staticDetails": [
                                {
                                        "invName": "",
                                        "address1": "",
                                        "address2": "",
                                        "address3": "",
                                        "city": "",
                                        "state": "",
                                        "pinCode": "",
                                        "mobile": "",
                                        "email": "",
                                        "stmtPeriod  ": ""
                                }
                        ]
                }
        ]
}
```

5. Common HTTP error codes

Code	Description
400	Validation error or Invalid request parameters or values
403	Session expired or invalidate.
429	Too many requests to the API (rate limiting)
500	Sever Connection error or Something unexpected went wrong

6. Appendix

```
package com.sspl.sample.filter;

import java.io.PrintWriter; import java.io.StringWriter;
import java.nio.charset.StandardCharsets; import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException; import java.security.spec.KeySpec;
import java.util.Base64; import java.util.Formatter; import java.util.Random;

import javax.crypto.Cipher;
import javax.crypto.NoSuchPaddingException; import javax.crypto.SecretKey;
import javax.crypto.SecretKeyFactory; import javax.crypto.spec.IvParameterSpec; import
javax.crypto.spec.PBEKeySpec; import javax.crypto.spec.SecretKeySpec;

import org.apache.commons.lang3.StringUtils; import org.jose4j.jwa.AlgorithmConstraints; import
org.jose4j.jwk.JsonWebKeySet;
import org.jose4j.jwk.RsaJsonWebKey; import org.jose4j.jws.AlgorithmIdentifiers; import
org.jose4j.jws.JsonWebSignature;
import org.jose4j.jwx.HeaderParameterNames; import org.json.JSONObject;
import org.springframework.http.HttpEntity; import org.springframework.http.HttpHeaders; import
org.springframework.http.ResponseEntity;
import org.springframework.util.LinkedMultiValueMap; import org.springframework.util.MultiValueMap;
import org.springframework.web.client.RestTemplate;


public class ThirdPartyUtilsCGS {

public static void main( String[] args) throws Exception{

// Below details will be shared by MFCentral during on-boarding String clientId = "";
String clientSecret = ""; String userName = ""; String password = "";
String encryptionDecryptionKey = ""; String privateKey = "";
String publicKey = "";

// Inputs to the request
String pan = ""; String pekrn = ""; String mobile = ""; String email = "";
String clientReferenceNumber = "";


// Request 1 - Token Generation
String tokenGenerationUrl = "https://devservices.mfcentral.com/oauth/token"; String token =
getToken(tokenGenerationUrl, clientId, clientSecret, userName, password);

System.out.println("Token : " + token );
```

```
// Request 2 - submit pan/pekrn & mobile/email to MFCentral for CGS generation. String request =
"{\"clientRefNo\": " + "\"" + clientReferenceNumber + "\""
+ ", \"pan\": " + "\"" + pan + "\""
+ ", \"pekrn\": " + "\"" + pekrn + "\""
+ ", \"mobile\": " + "\"" + mobile + "\""
+ ", \"email\" : " + "\"" + email + "\""
+ " }";


String cgsRequestUrl = "https://devservices.mfcentral.com/api/client/V1/submitstatementrequest";
JSONObject response = submitRequestToMFCentral(cgsRequestUrl, token, clientId, encryptionDecryptionKey,
privateKey, publicKey, request);

// Request 3 - Submit investor Consent to MFCentral String otpValidationUrl =
"https://devservices.mfcentral.com/api/client/V1/investorconsent";
JSONObject otpRequest = new
JSONObject("{\"reqId\":55718,\"otpRef\":\"35979fc2-e2c8-4faa-b6dc-
ab6505a4612e\",\"userSubjectReference\":\"\",\"clientRefNo\":\"testing12345\"}");
otpRequest.put("enteredOtp", "574224");

submitRequestToMFCentral(otpValidationUrl, token, clientId, encryptionDecryptionKey, privateKey,
publicKey, otpRequest.toString());

// Request 4 - Get CGS Document String cgsDocumentUrl =
"https://devservices.mfcentral.com/api/client/V1/getstatementdocument";
JSONObject cgsDocument = submitRequestToMFCentral(cgsDocumentUrl, token, clientId,
encryptionDecryptionKey, privateKey, publicKey, otpRequest.toString());

System.out.println("CGS: " + cgsDocument);
}

public static String generateSignature(String payload, String privateKey) {
 JSONObject output = new JSONObject();

try {

RsaJsonWebKey rsaJsonWebKey = getPrivateKey(privateKey);
 JsonWebSignature jws = new JsonWebSignature();
 jws.setAlgorithmHeaderValue(AlgorithmIdentifiers.RSA_USING_SHA256);
jws.setKeyIdHeaderValue(rsaJsonWebKey.getKeyId());


jws.getHeaders().setObjectHeaderValue(HeaderParameterNames.BASE64URL_ENCODE_P AYLOAD, false);
jws.setPayload(payload); jws.setKey(rsaJsonWebKey.getPrivateKey());
String digitalSignature = jws.getDetachedContentCompactSerialization();
output = new JSONObject();
output.put("request", payload);
output.put("signature", digitalSignature);

return output.toString();
} catch (Exception ex) {
ex.printStackTrace();
}

return null;
}
```

```
public static boolean verifyDigitalSignature(String payload, String digitalSignature, String publicKey) {

try {
RsaJsonWebKey rsaJsonWebKey = getPublicKey(publicKey);

JsonWebSignature verifierJws = new JsonWebSignature();
verifierJws.setAlgorithmConstraints(new
AlgorithmConstraints(AlgorithmConstraints.ConstraintType.WHITELIST,
AlgorithmIdentifiers.RSA_USING_SHA256));
verifierJws.setKey(rsaJsonWebKey.getPublicKey());
verifierJws.setCompactSerialization(digitalSignature);
verifierJws.setPayload(payload);
return verifierJws.verifySignature();

} catch (Exception ex) {
ex.printStackTrace();
}

return false;
}

public static RsaJsonWebKey getPrivateKey(String privateKey) throws Exception { JsonWebKeySet
jsonWebKeySet = new JsonWebKeySet(privateKey); RsaJsonWebKey rsaJsonWebKey = (RsaJsonWebKey)
jsonWebKeySet.getJsonWebKeys().get(0);
return rsaJsonWebKey;
}

public static RsaJsonWebKey getPublicKey(String publicKey) throws Exception{
JSONObject jsonObject = new JSONObject(publicKey);
RsaJsonWebKey rsaJsonWebKey = new RsaJsonWebKey(jsonObject.toMap()); return rsaJsonWebKey;
}

public static String getToken(String tokenGenerationUrl, String clientId, String clientSecret, String userName,
String password) {

Base64.Encoder encoder = Base64.getEncoder(); HttpHeaders headers = new HttpHeaders();
headers.add("Content-Type", "application/x-www-form-urlencoded"); headers.add("Authorization", "Basic " +
encoder.encodeToString((clientId+":"+clientSecret).getBytes()));

MultiValueMap<String, String> map= new LinkedMultiValueMap<String, String>(); map.add("username",
userName);
map.add("password", password); map.add("grant_type", "password");

HttpEntity<MultiValueMap<String, String>> request = new HttpEntity<MultiValueMap<String, String>>(map,
headers);

RestTemplate restTemplate = new RestTemplate(); ResponseEntity<String> response =
restTemplate.postForEntity(
tokenGenerationUrl, request , String.class );
String responseBody = response.getBody();

JSONObject json = new JSONObject(responseBody); return json.getString("access_token");
}

public static JSONObject submitRequestToMFCentral(String url, String token, String clientId, String
encryptionKey, String privateKey, String publicKey, String payload) throws Exception {
```

```
String encryptedText = CryptUtils.encryptDecrypt("ENCRYPT", payload, encryptionKey);
String mfcRequest = generateSignature(encryptedText, privateKey);
HttpHeaders headers = new HttpHeaders(); headers.add("Content-Type", "application/json");
headers.add("ClientId", clientId); headers.add("Authorization", "Bearer " + token);

RestTemplate restTemplate = new RestTemplate();
HttpEntity<String> request = new HttpEntity<String>(mfcRequest, headers); ResponseEntity<String> response
= restTemplate.postForEntity( url, request, String.class );


String responseBody = response.getBody(); JSONObject json = new JSONObject(responseBody);

boolean isVerified = verifyDigitalSignature(json.getString("response"), json.getString("signature"), publicKey);

if( ! isVerified ) {
return null;
}

String decryptedText = CryptUtils.encryptDecrypt("DECRYPT", json.getString("response"), encryptionKey);


System.out.println("Response from MFCentral: " + decryptedText);

if( decryptedText == null || decryptedText.trim().length() == 0 ) { return null;
}

return new JSONObject(decryptedText);
}
}

class CryptUtils
{

private static String TRANSFORMATION = "AES/CBC/PKCS5Padding"; private static String ALGORITHM = "AES";
private static String DIGEST = "SHA-256";

private static Cipher _cipher; private static SecretKey _password;
private static IvParameterSpec _IVParamSpec;

//16-byte private key
private static byte[] iv = "globalaesvectors".getBytes();

public static String encryptDecrypt(String Mode, String cryptText, String passKey) throws Exception {
byte[] decryptedVal = null; String retText="";
try {

//Encode digest MessageDigest digest;
digest = MessageDigest.getInstance(DIGEST);
byte[] bytehash = digest.digest(passKey.getBytes(StandardCharsets.UTF_8));

String strHash="";
Formatter formatter = new Formatter();

for (byte myByte: bytehash) { strHash="";
strHash = String.format("%02x", myByte); formatter.format(StringUtils.leftPad(strHash, 2, "0"));
```

```
}
strHash = formatter.toString();

if (strHash.length() > 32) {
strHash = strHash.substring(0, 32);
}
byte[] bytekeytemp = strHash.getBytes(StandardCharsets.UTF_8); byte[] bytekey = new byte[32];
System.arraycopy(bytekeytemp, 0, bytekey, 0, 32);

_password = new SecretKeySpec(bytekey, ALGORITHM);


//Initialize objects
_cipher = Cipher.getInstance(TRANSFORMATION);
_IVParamSpec = new IvParameterSpec(iv);

if (Mode.equals("ENCRYPT")){
_cipher.init(Cipher.ENCRYPT_MODE, _password, _IVParamSpec);
byte[] ciphertext = _cipher.doFinal(cryptText.getBytes(StandardCharsets.UTF_8));

byte[] encodedValue = Base64.getEncoder().encode(ciphertext); String encryptedVal = new
String(encodedValue);
encryptedVal = encryptedVal.replace("+", "-"); encryptedVal = encryptedVal.replace("/", "_"); retText =
encryptedVal;
}
else {
_cipher.init(Cipher.DECRYPT_MODE, _password, _IVParamSpec);

cryptText = cryptText.replace("-", "+");
cryptText = cryptText.replace("_", "/");

byte[] decodedValue = Base64.getDecoder().decode(cryptText.getBytes(StandardCharsets.UTF_8));
decryptedVal = _cipher.doFinal(decodedValue); retText = new String(decryptedVal);
}

} catch (NoSuchAlgorithmException e) {
throw new Exception("Encryption error, No such algorithm ->" + ALGORITHM);
} catch (NoSuchPaddingException e) {
throw new Exception("Encryption error, No such padding PKCS7");
} catch (Exception e) {
throw new Exception("Encryption error:" + exceptionToString(e));
}
return retText;
}


private static final String SECRET_KEY = "crKZelC5VNbowCKFTwGIdCf8usJD";
//private static final String SALT = "ssshhhhhhhhhhh!!!!";
public static String encrypt(String strToEncrypt, String salt) throws Exception {
try {
byte[] iv = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
IvParameterSpec ivspec = new IvParameterSpec(iv);

SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
KeySpec spec = new PBEKeySpec(SECRET_KEY.toCharArray(), salt.getBytes(), 1000, 256);
SecretKey tmp = factory.generateSecret(spec);
SecretKeySpec secretKey = new SecretKeySpec(tmp.getEncoded(), "AES");
```

```
Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding"); cipher.init(Cipher.ENCRYPT_MODE, secretKey,
ivspec);
return Base64.getEncoder()
.encodeToString(cipher.doFinal(strToEncrypt.getBytes(StandardCharsets.UTF_8)));
} catch (Exception e) {
throw new Exception("Encryption error:" + exceptionToString(e));

}
}

public static String decrypt(String strToDecrypt, String salt) throws Exception {
try {
byte[] iv = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
IvParameterSpec ivspec = new IvParameterSpec(iv);

SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
KeySpec spec = new PBEKeySpec(SECRET_KEY.toCharArray(), salt.getBytes(), 1000, 256);
SecretKey tmp = factory.generateSecret(spec);
SecretKeySpec secretKey = new SecretKeySpec(tmp.getEncoded(), "AES");

Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5PADDING");
cipher.init(Cipher.DECRYPT_MODE, secretKey, ivspec);
return new String(cipher.doFinal(Base64.getDecoder().decode(strToDecrypt)));
} catch (Exception e) {
throw new Exception("Decryption error:" + exceptionToString(e));
}
}

public static String exceptionToString(Exception ex) {
StringWriter writer      = new StringWriter();
PrintWriter printWriter = new PrintWriter(writer);

ex.printStackTrace(printWriter);
return writer.toString();
}

public static String randomKey() {
/*
*       Random r = new SecureRandom(); byte[] salt = new byte[20]; r.nextBytes(salt);
*       return new String(salt,Charset.forName("UTF-8"));
*/

int leftLimit = 48; // numeral '0'
int rightLimit = 122; // letter 'z'
int targetStringLength = 10;
Random random = new Random();

String generatedString = random.ints(leftLimit, rightLimit + 1)
.filter(i -> (i <= 57 || i >= 65) && (i <= 90 || i >= 97))
.limit(targetStringLength)
.collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
.toString();

return generatedString;
}
```

```
public static String bytesToHex(byte[] hash) {
StringBuilder hexString = new StringBuilder(2 * hash.length);
for (int i = 0; i < hash.length; i++) {
String hex = Integer.toHexString(0xff & hash[i]); if(hex.length() == 1) {
hexString.append('0');
}


hexString.append(hex);
}
return hexString.toString();
}

public static byte[] getSHA(String input, String salt) throws NoSuchAlgorithmException
{
// Static getInstance method is called with hashing SHA MessageDigest md =
MessageDigest.getInstance("SHA-256");

// Add password bytes to digest md.update(salt.getBytes());

// digest() method called
// to calculate message digest of an input
// and return array of byte
return md.digest(input.getBytes(StandardCharsets.UTF_8));
```